

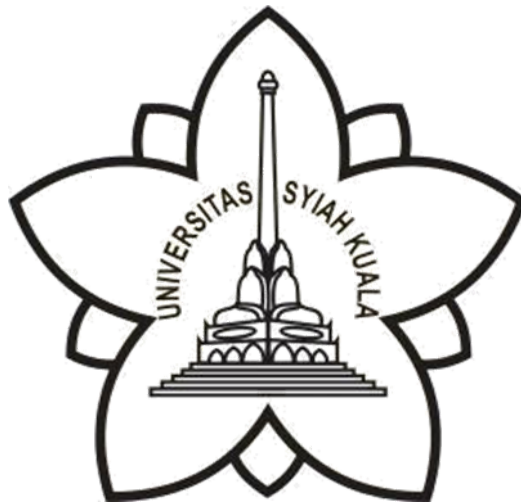
TUGAS 3

Logistic Regression, K-Nearest Neighbor, Naïve Bayes, Decision Tree

disusun untuk memenuhi
tugas mata kuliah Pembelajaran Mesin Kelas A

Oleh:

Aulia Vika Rahman	2208107010001
Shofia Nurul Huda	2208107010015
M. Ihsan Rizqullah Adfa	2208107010029
Nashed Bayazid Ali	2208107010049
Muhammad Raza Adzani	2208107010066
T. Farhan Syah	2208107010090



DEPARTEMEN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
TAHUN 2025

1. PENDAHULUAN

a. Latar Belakang

Kesehatan maternal menjadi indikator penting dalam kesehatan masyarakat global. WHO melaporkan bahwa sekitar 810 wanita meninggal setiap hari akibat komplikasi kehamilan yang sebenarnya dapat dicegah. Mayoritas kasus terjadi di negara berkembang dengan akses kesehatan terbatas. Identifikasi dini ibu hamil berisiko tinggi menjadi kunci dalam menurunkan angka kematian maternal.

Kemajuan teknologi informasi dan kecerdasan buatan, khususnya machine learning, membuka peluang besar untuk meningkatkan deteksi dini faktor risiko kehamilan. Machine learning dapat menganalisis berbagai parameter kesehatan secara simultan dan mengidentifikasi pola yang mungkin terlewatkan pada metode konvensional.

Kehamilan menyebabkan berbagai perubahan fisiologis seperti tekanan darah, kadar gula darah, detak jantung, dan suhu tubuh. Parameter ini, bersama faktor demografis seperti usia, menjadi indikator penting tingkat risiko kehamilan. Dengan menganalisis parameter tersebut menggunakan algoritma machine learning, dapat dikembangkan sistem prediksi untuk membantu tenaga medis dalam pengambilan keputusan klinis.

b. Tujuan

1. Mengembangkan model klasifikasi untuk memprediksi tingkat risiko kesehatan maternal berdasarkan parameter fisiologis dan demografis
2. Mengidentifikasi faktor-faktor yang berkontribusi terhadap tingkat risiko kesehatan maternal
3. Membandingkan performa empat algoritma machine learning (K-Nearest Neighbors, Decision Tree, Logistic Regression, dan Naive Bayes) dalam klasifikasi risiko maternal
4. Menentukan model terbaik untuk implementasi dalam sistem pendukung keputusan klinis

2. PEMAHAMAN DATASET

a. Sumber Data

Dataset yang digunakan adalah "Maternal Health Risk Data Set" yang berisi data kesehatan ibu hamil beserta klasifikasi tingkat risiko kehamilan. Dataset

ini digunakan untuk memprediksi risiko kesehatan maternal berdasarkan beberapa parameter fisiologis. Dataset ini kami dapat di situs UCI dengan url : <https://archive.ics.uci.edu/dataset/863/maternal+health+risk> , dataset ini juga telah di pakai dalam beberapa artikel ilmiah.

b. Variabel yang digunakan

Dataset ini terdiri dari 7 kolom, dengan 6 fitur independen dan 1 target:

- Age: Usia wanita hamil (tahun)
- SystolicBP: Tekanan darah sistolik (mmHg)
- DiastolicBP: Tekanan darah diastolik (mmHg)
- BS: Kadar glukosa darah (mmol/L)
- BodyTemp: Suhu tubuh
- HeartRate: Detak jantung istirahat (bpm)
- RiskLevel: Target/variabel dependen dengan 3 kelas - "low risk", "mid risk", dan "high risk"

c. Statistik Deskriptif dan Visualisasi Awal

Dataset awal terdiri dari 1014 entri dengan 7 fitur. Setelah dilakukan pengecekan awal, ditemukan bahwa:

- Tidak terdapat nilai null dalam dataset
- Terdapat banyak baris duplikat
- Rentang usia berkisar dari 10 hingga 70 tahun
- Semua kolom memiliki nilai minimum > 0
- Terdapat anomali pada kolom HeartRate dengan nilai 7, yang kemungkinan besar adalah kesalahan input

3. EKSPLORASI DATA DAN PRA-PEMROSESAN

a. Pembersihan Data

1. Penanganan Data Duplikat

```
df = df.drop_duplicates().reset_index(drop=True)
```

```
df.shape
```

```
(452, 7)
```

gambar 3.1.1. Penanganan Data Duplikat

Pada tahapan ini, data duplikat dihapus, dan menghasilkan dataset final dengan 452 observasi

2. Perbaiki Nilai Tidak Masuk Akal

Terdapat sebuah observasi dengan nilai heart rate sebesar 7, yang tidak masuk akal. Kemungkinan besar, ini adalah kesalahan saat entri data. Namun, kita tidak mengetahui nilai yang sebenarnya — bisa saja seharusnya 70, 67, atau 77. Oleh karena itu, kita akan menggantinya dengan modus (nilai yang paling sering muncul) dari kolom tersebut.

```
[ ] df.HeartRate.mode()
```

```
0    70  
Name: HeartRate, dtype: int64
```

```
[ ] df.loc[df.HeartRate == 7, "HeartRate"] = 70
```

gambar 3.1.2. Perbaiki Nilai Tidak Masuk Akal

- Ditemukan nilai tidak masuk akal pada kolom HeartRate (nilai 7)
- Nilai tersebut diganti dengan modus (70) dari kolom HeartRate

3. Encoding Fitur Kategorikal

```
[ ] df.RiskLevel.unique()
```

```
array(['high risk', 'low risk', 'mid risk'], dtype=object)
```

gambar 3.1.3.1. Pengecekan nilai unik kolom RiskLevel

Tipe kolom RiskLevel adalah ordinal, dengan kata lain, nilainya berada dalam kategori yang berurutan. Untuk itu, kita dapat melakukan label encoding, di mana setiap nilai akan secara manual dipetakan ke kunci yang sesuai dengan menggunakan fungsi `replace()`.

```
[ ] df.replace({"high risk":2, "mid risk":1, "low risk":0}, inplace=True)  
df.head()
```

```
C:\Users\HP VICTUS\AppData\Local\Temp\ipykernel_23340\247815498.py:1: Fut  
df.replace({"high risk":2, "mid risk":1, "low risk":0}, inplace=True)
```

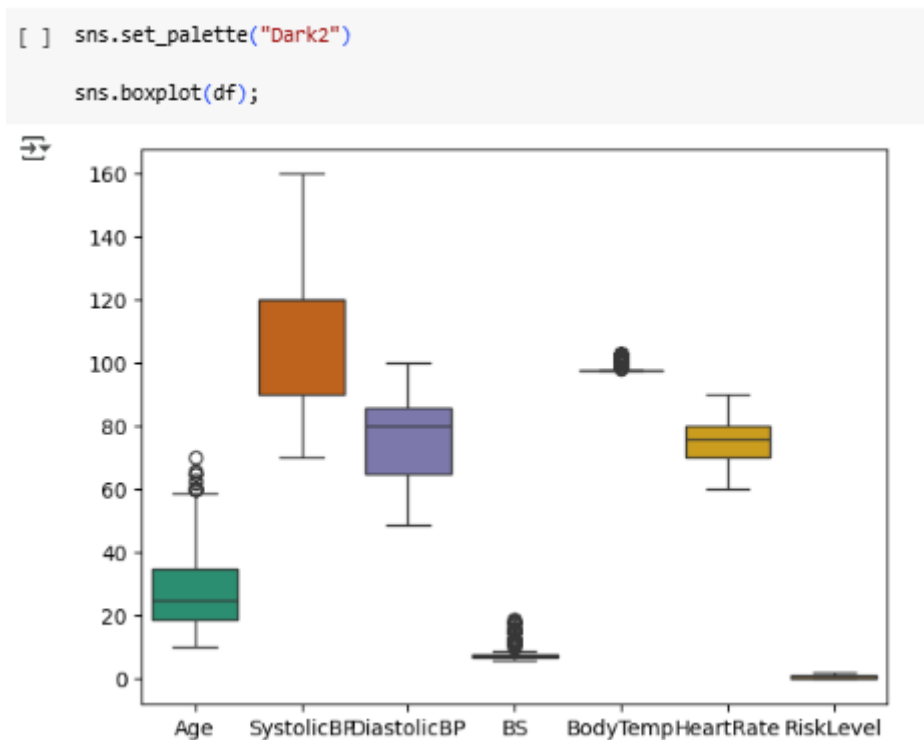
	Age	SystolicBP	DiastolicBP	BS	BodyTemp	HeartRate	RiskLevel
0	25	130	80	15.0	98.0	86	2
1	35	140	90	13.0	98.0	70	2
2	29	90	70	8.0	100.0	80	2
3	30	140	85	7.0	98.0	70	2
4	35	120	60	6.1	98.0	76	0

gambar 3.1.3.2. Encoding fitur kategorikal

b. Analisis Outlier

Dalam statistik, outlier adalah titik observasi yang jauh dari observasi lainnya. Outlier dapat disebabkan oleh kesalahan dalam pengumpulan atau pencatatan data, atau karena variabilitas tinggi secara alami pada titik data. Cara mengatasi outlier sangat bergantung pada data kita atau jenis analisis yang akan dilakukan. Outlier dapat secara signifikan memengaruhi model kita dan juga bisa menjadi sumber informasi yang berharga, memberikan wawasan tentang perilaku tertentu.

Ada banyak cara untuk menemukan outlier dalam data kita. Kita bisa melakukan analisis univariat (menggunakan analisis satu variabel) atau analisis multivariat (menggunakan dua atau lebih variabel). Salah satu cara paling sederhana untuk mendeteksi outlier adalah dengan memeriksa data secara visual, dengan membuat box plot.

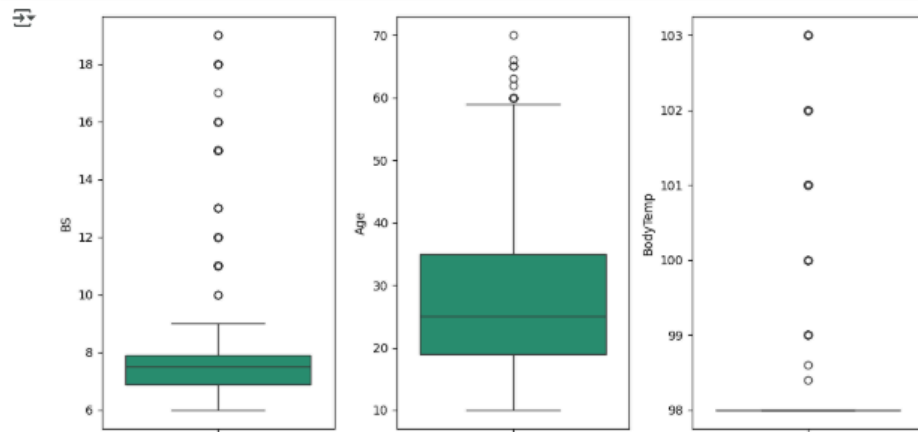


gambar 3.2.1 Analisis Outlier

Dari boxplot sederhana dari seluruh dataframe di atas, kita dapat menemukan kolom-kolom yang memiliki outlier untuk pemeriksaan lebih lanjut. Selain itu, kita juga dapat melihat bahwa variabel-variabel kita memiliki skala yang berbeda, dan nantinya kita mungkin perlu melakukan feature scaling.

```
[ ] plt.figure(figsize=(10, 5))
plt.subplot(1,3,1)
sns.boxplot(y = 'BS', data = df)
plt.subplot(1,3,2)
sns.boxplot(y = 'Age', data = df)
plt.subplot(1,3,3)
sns.boxplot(y = 'BodyTemp', data = df)

plt.tight_layout()
plt.show();
```



gambar 3.2.2. Analisis Outlier

Seperti yang dapat kita lihat dari plot ini, terdapat beberapa titik yang terletak di luar area box plot dan sangat menyimpang dari sisa populasi. Apakah kita akan menghapus atau mempertahankan titik-titik tersebut sangat bergantung pada pemahaman kita tentang data dan jenis analisis yang akan dilakukan. Dalam kasus ini, titik-titik yang berada di luar box plot kita mungkin merupakan titik data yang benar-benar valid dan tidak perlu dihapus. Outlier diidentifikasi menggunakan visualisasi boxplot pada beberapa variabel:

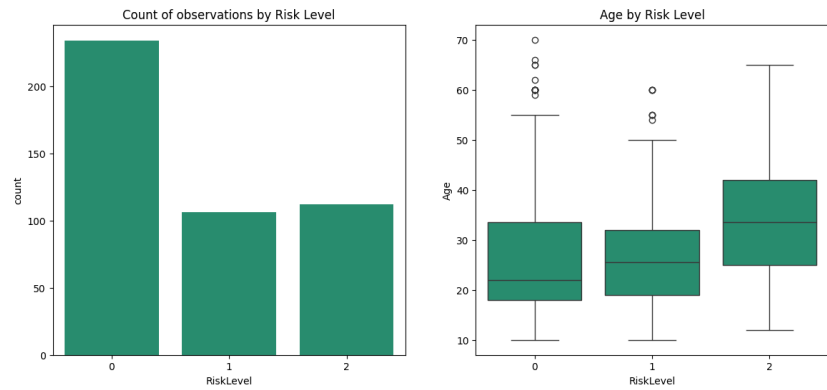
- Ditemukan outlier pada variabel BS (kadar gula darah)
- Beberapa outlier terlihat pada variabel Age
- Terdapat beberapa outlier pada variabel BodyTemp

Outlier dipertahankan karena dianggap sebagai data valid yang mewakili kasus yang memang ekstrem namun penting dalam konteks kesehatan ibu hamil.

c. Analisis Eksplorasi Data

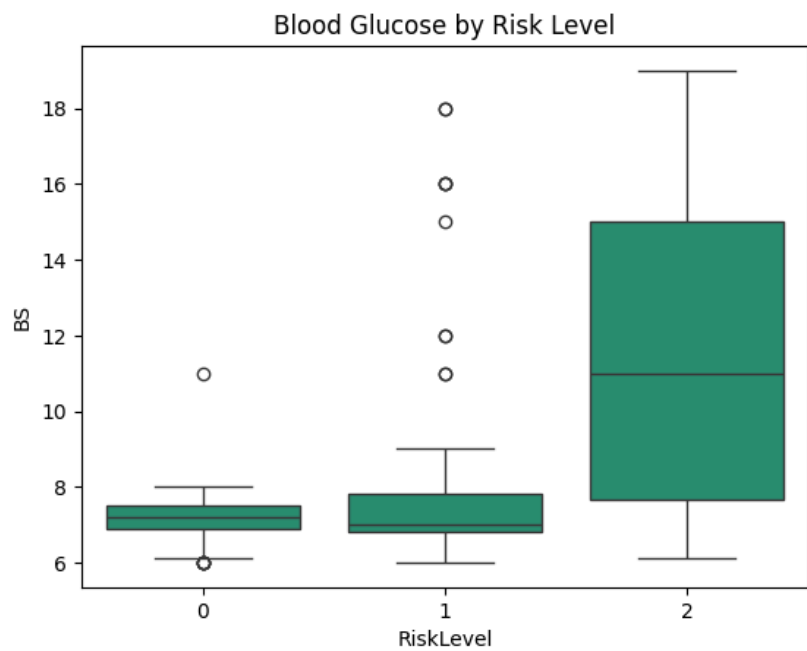
1. Analisis Univariat dan Bivariat

- Distribusi Risiko Kehamilan dan Hubungan Risiko Kehamilan berdasarkan usia



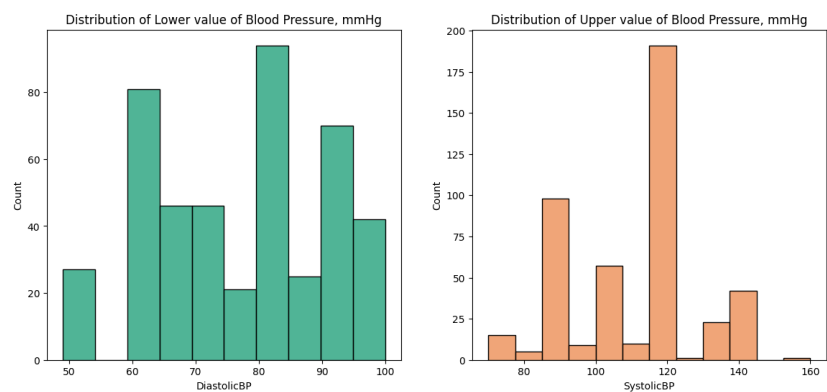
gambar 3.3.1. Distribusi risiko kehamilan

- Kadar Gula Darah

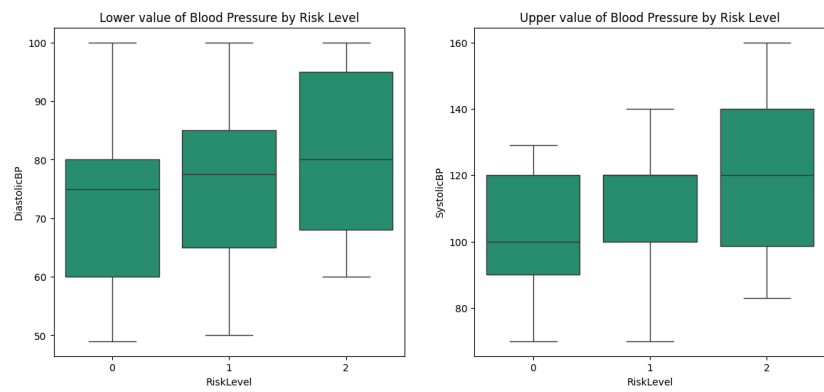


gambar 3.3.2. distribusi kadar gula darah berdasarkan risiko kehamilan

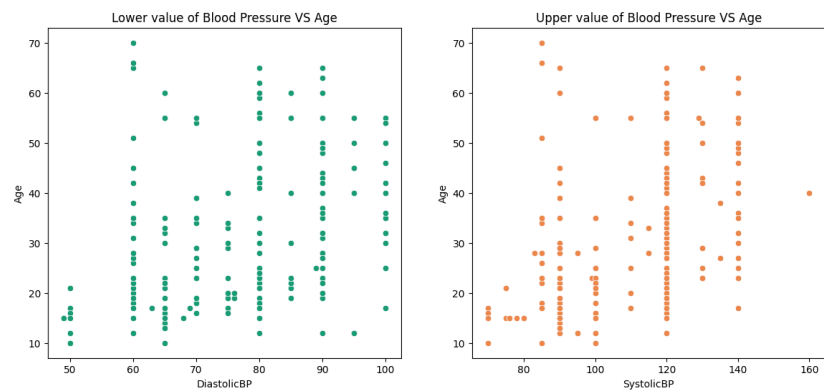
- Tekanan Darah



gambar 3.3.3. Distribusi tekanan darah

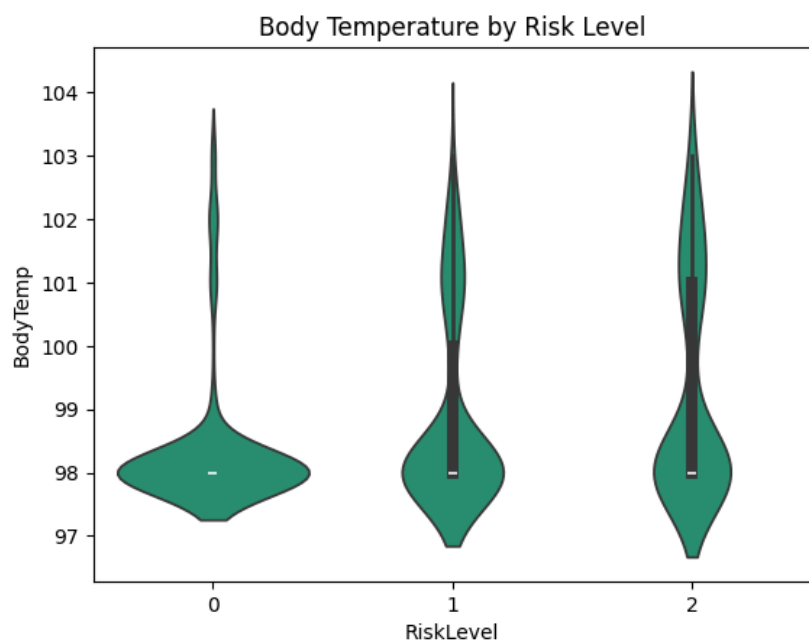


gambar 3.3.4. analisis tekanan darah berdasarkan risiko kehamilan

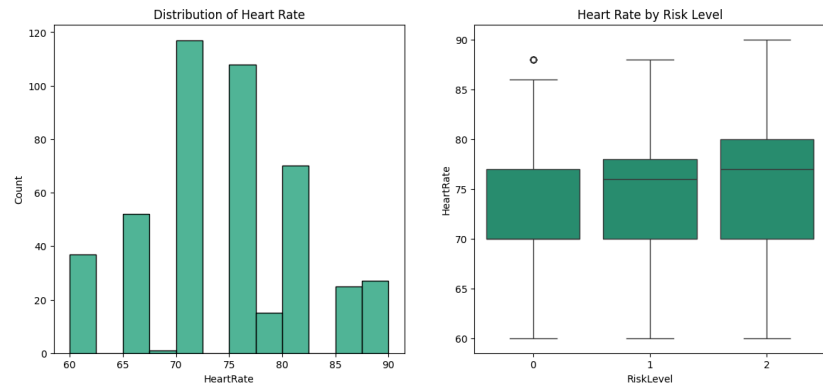


gambar 3.3.5. analisis tekanan darah berdasarkan umur

- Suhu Tubuh



- Detak Jantung



gambar 3.3.6. distribusi detak jantung

Observasi hasil yang kami dapatkan dari visualisasi yang telah kami lakukan:

- Kehamilan dengan risiko rendah adalah yang paling sering terjadi secara keseluruhan, terjadi pada lebih dari separuh kasus.
- Wanita yang lebih muda cenderung memiliki kehamilan dengan risiko rendah dan sedang, sementara kehamilan pada wanita di atas usia 35 tahun lebih sering diklasifikasikan sebagai risiko tinggi, sehingga memerlukan perhatian lebih.
- Jika seorang wanita hamil memiliki kadar gula darah lebih dari 8 mmol/L, dalam sebagian besar kasus, kehamilan tersebut dianggap berisiko tinggi.
- Distribusi nilai tekanan darah bawah lebih tersebar, sekitar 60-100 mmHg, dibandingkan dengan distribusi nilai tekanan darah atas yang lebih terpusat di sekitar 120 mmHg.
- Tekanan darah yang lebih tinggi (baik sistolik maupun diastolik) dan suhu tubuh yang lebih tinggi terkait dengan kehamilan berisiko tinggi.
- Mengenai korelasi antara usia dan tekanan darah, tekanan darah yang sangat rendah (baik sistolik maupun diastolik) terlihat pada beberapa gadis dan wanita muda, tetapi tekanan darah normal dan tinggi tampaknya tidak

terlalu berkorelasi dengan usia, setidaknya pada dataset kita.

- Heart rate wanita hamil terdistribusi normal dan hanya sedikit terkait dengan tingkat risiko kehamilan.

2. Analisis Korelasi

Analisis korelasi sangat berguna untuk membangun korelasi antara variabel respon dan variabel prediktor lainnya, karena beberapa di antaranya mungkin tidak memiliki dampak signifikan dalam menentukan tingkat risiko dan tidak akan digunakan dalam analisis. Disini, kami membuat heatmap(), menggunakan library seaborn.

```
[ ] plt.figure(figsize=(12, 10))
    sns.heatmap(df.corr(),annot=True,cmap='RdYlGn', vmin=-1, center=0,
                  cbar_kws={"shrink": .5})

    plt.show()
```



gambar 3.3.7.

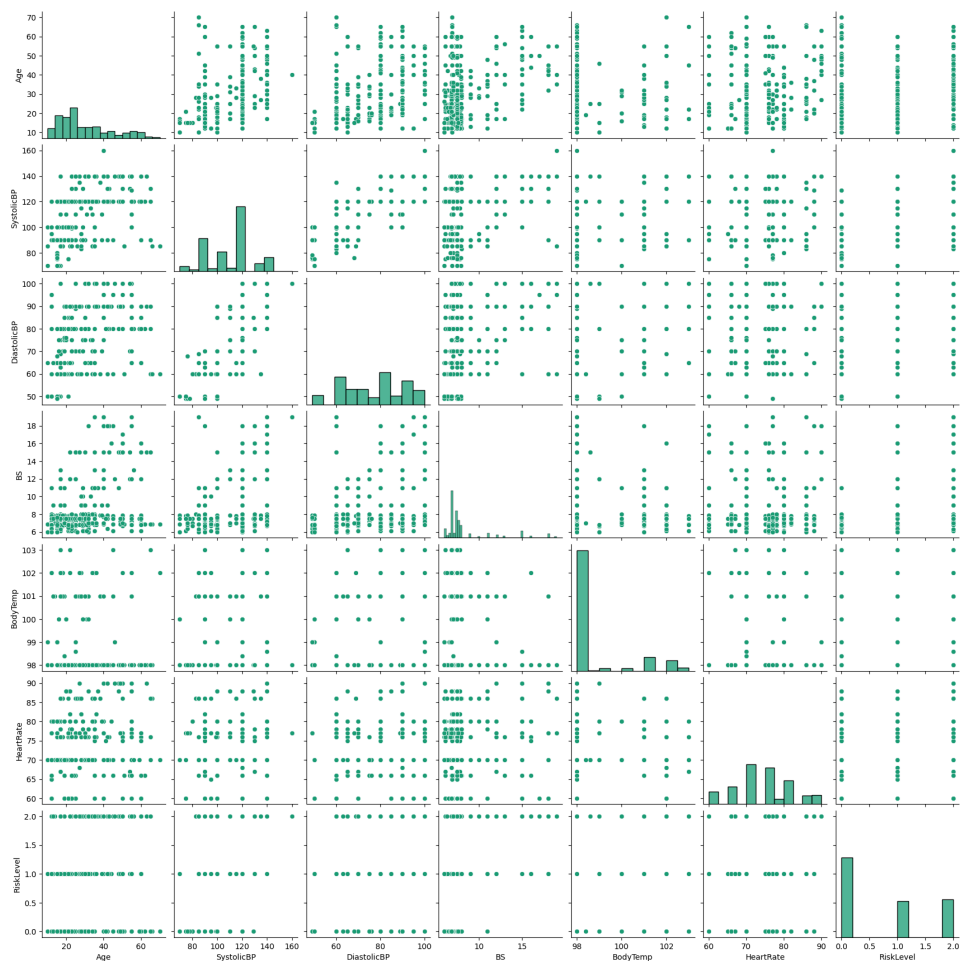
Dari heatmap di atas, hijau ekstrem berarti fitur yang sangat berkorelasi positif (hubungan antara dua variabel di mana kedua variabel bergerak dalam arah yang sama), sedangkan merah ekstrem berarti fitur yang

berkorelasi negatif (hubungan antara dua variabel di mana peningkatan satu variabel dikaitkan dengan penurunan variabel lainnya). Semua variabel tampaknya memiliki korelasi positif dengan variabel respon kita.

Kita dapat melihat bahwa hanya ada satu variabel yang sangat berkorelasi, yaitu BS (gula darah). Variabel lainnya memiliki korelasi positif, tetapi tidak begitu kuat. Jika kita memiliki banyak variabel, kita bisa memilih hanya variabel yang sangat berkorelasi untuk analisis lebih lanjut. Namun, karena kita hanya memiliki 7 kolom, kita akan menggunakan semua variabel tersebut.

d. Transformasi Fitur untuk mengatasi Skewness

Skewness mengacu pada distorsi data atau simetri/asimetri distribusi data. Ketika data Anda terdistorsi (skewed), nilai mean dan median akan berbeda. Skewness melanggar asumsi normalitas pada beberapa model machine learning, misalnya regresi linier.



gambar 3.4.1. Pengecekan variabel terdistorsi

```
[ ] skew_limit = 0.75 # define a limit above which we will log transform

# Create a list of numerical cols to check for skewing
mask = data.dtypes != object
num_cols = data.columns[mask]

skew_vals = df[num_cols].skew()

[ ] # Showing the skewed columns
skew_cols = (skew_vals
              .sort_values(ascending=False)
              .to_frame()
              .rename(columns={0: 'Skew'})
              .query('abs(Skew) > {}'.format(skew_limit)))

skew_cols
```

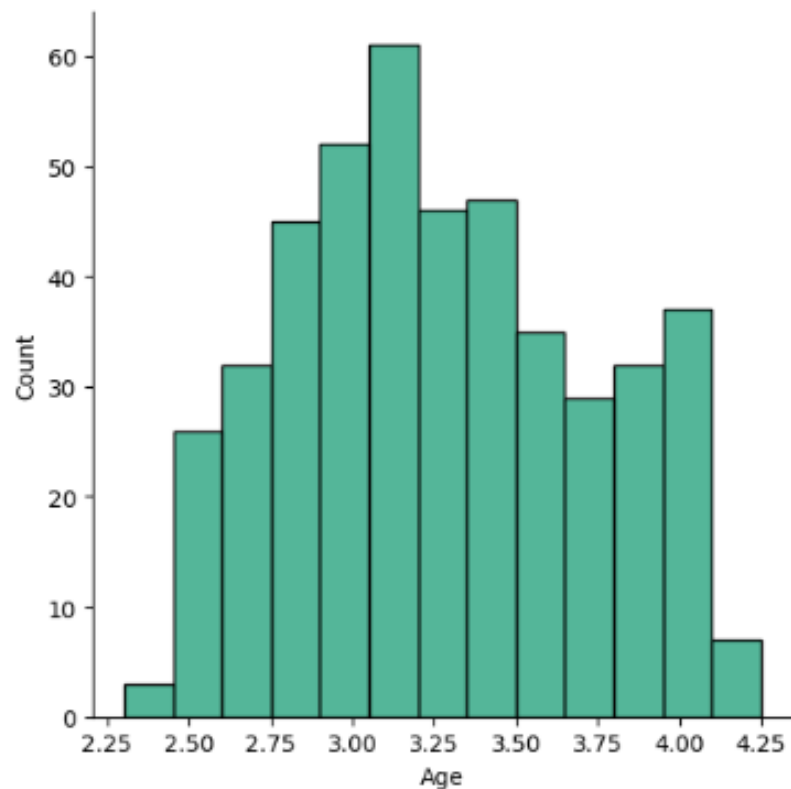
	Skew
BS	2.262874
BodyTemp	1.751794
Age	0.922079

gambar 3.4.2. Menghitung tingkat skewness

```
[ ] df_transformed = df.copy()

df_transformed["Age"] = df["Age"].apply(np.log)
print(df_transformed["Age"].skew())
sns.displot(df_transformed.Age);
```

0.197419127736239



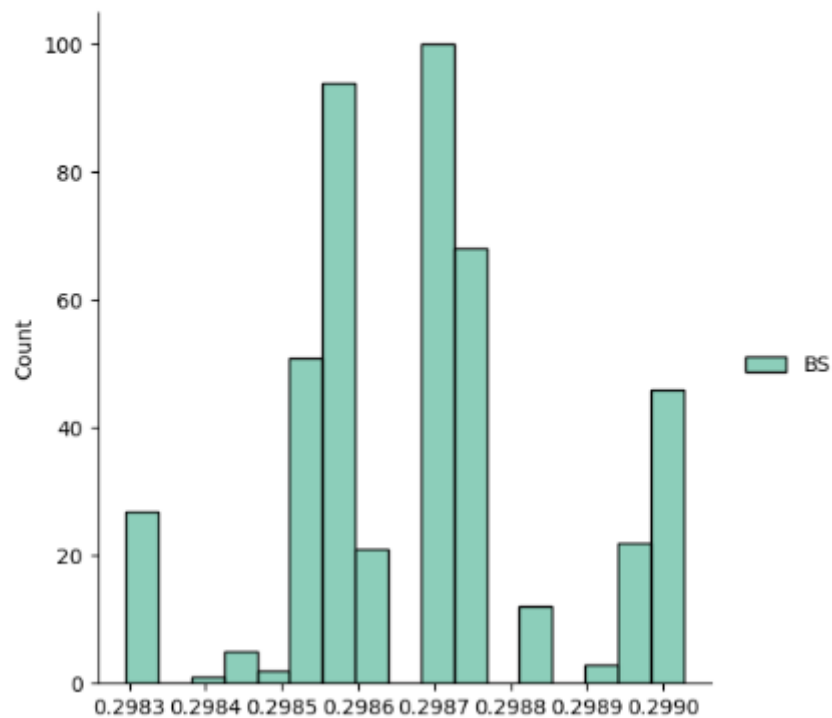
```
[ ] bc_result = boxcox(df.BS)
    boxcox_bs = pd.DataFrame(bc_result[0], columns=['BS'])
    lambd = bc_result[1]
```

```
[ ] lambd
```

```
→ -3.3440041657535655
```

```
[ ] print(boxcox_bs.skew())
    sns.displot(boxcox_bs);
```

```
→ BS    0.264687
    dtype: float64
```



```
[ ] df_transformed['BS'] = boxcox_bs['BS']
    df_transformed[['BS', 'Age']].skew().to_frame().rename(columns=
        {0: 'Skew'}).sort_values('Skew', ascending=False)
```

```
→
      Skew
BS  0.264687
Age  0.197419
```

gambar 3.4.3. Melakukan transformasi log dan box cox

Setelah melakukan transformasi log dan Box-Cox, skewness dari:

- Gula darah berkurang dari 2.26 menjadi 0.26.
- Umur berkurang dari 0.92 menjadi 0.2.

Ini membuat distribusinya menjadi cukup normal.

e. Persiapan Data untuk Pemodelan

1. Pemisahan Fitur dan Target

Menetapkan variabel X sama dengan fitur numerik dan variabel y sama dengan kolom "RiskLevel".

```
[ ] x = df_transformed.drop('RiskLevel', axis=1)
    y = df_transformed['RiskLevel']
```

gambar 3.5.1. Memisahkan fitur dan target

- Fitur (X): semua kolom kecuali RiskLevel
- Target (y): kolom RiskLevel

2. Standarisasi Fitur

Menormalkan skala variabel menggunakan StandardScaler.

```
[ ] scaler = StandardScaler()
    scaler.fit(X)
    X_scaled = scaler.transform(X)
```

gambar 3.5.2. Feature scaling

4. IMPLEMENTASI MODEL

a. Data Splitting

```
[ ] x_train, x_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
                                                         random_state=101, stratify=y)
```

gambar 4.1. Data splitting

Dataset dibagi menjadi data training (70%) dan data testing (30%) dengan stratifikasi pada variabel target untuk memastikan distribusi kelas yang seimbang.

b. Algoritma yang Diimplementasikan

1. K-Nearest Neighbors (KNN)

```
[ ] error_rate = []

for i in range(1,20):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train,y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))

[ ] plt.figure(figsize=(10,6))
plt.plot(range(1,20), error_rate, color='blue', linestyle='dashed', marker='o',
        markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate');

[ ] knn = KNeighborsClassifier(n_neighbors=16)

knn.fit(X_train,y_train)
knn_preds = knn.predict(X_test)

print('K Nearest Neighbors K=16')
print('\n')
print(confusion_matrix(y_test,knn_preds))
print('\n')
print(classification_report(y_test,knn_preds))
```

gambar 4.2.1. implementasi KNN

- Hyperparameter tuning untuk nilai optimal K (1-19)
- K optimal = 16 berdasarkan error rate

2. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix, classification_report

# 1. Membuat model Decision Tree dasar
tree = DecisionTreeClassifier(random_state=122)

# 2. Definisikan hyperparameter grid untuk tuning
param_grid = {
    'criterion': ['gini', 'entropy'],          # fungsi pengukuran
    'splitter': ['best', 'random'],            # strategi pemilihan
    'max_depth': [10, 20, 30],                 # kedalaman maksimal
    'min_samples_split': [2, 5, 10],           # minimal jumlah samples
    'min_samples_leaf': [2, 4]                 # minimal jumlah samples
}
```

```

# 3. Setup GridSearchCV
grid_search_tree = GridSearchCV(
    estimator=tree,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# 4. Fit Grid Search ke data training
grid_search_tree.fit(X_train, y_train)

# 5. Print best parameters dan best score
print('Best Parameters: ', grid_search_tree.best_params_)
print('Best Cross-Validation Accuracy: ', grid_search_tree.best_score_)

# 6. Gunakan model terbaik untuk prediksi di test set
best_tree = grid_search_tree.best_estimator_

# Prediksi di test set
tree_preds = best_tree.predict(X_test)

# 7. Evaluasi model
print('Decision Tree Performance on Test Set')
print('\n')
print(confusion_matrix(y_test, tree_preds))
print('\n')
print(classification_report(y_test, tree_preds))

```

gambar 4.2.2. implementasi decision tree

Grid search untuk hyperparameter tunin:

- criterion: gini, entropy
- splitter: best, random
- max_depth: 10, 20, 30
- min_samples_split: 2, 5, 10
- min_samples_leaf: 2, 4

3. Logistic Regression

```

● ● ●

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix, classification_report

# Membuat model dasar
logreg = LogisticRegression(multi_class='multinomial', solver='saga',
                             max_iter=5000, random_state=122)

# Definisikan grid hyperparameter
param_grid_logreg = {
    'C': [0.01, 0.1, 1, 10, 100], # regularization strength
    'penalty': ['l1', 'l2'], # regularization type
}

```



```

# Setup GridSearchCV
grid_search_logreg = GridSearchCV(
    estimator=logreg,
    param_grid=param_grid_logreg,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit ke training data
grid_search_logreg.fit(X_train, y_train)

# Print hasil terbaik
print('Best Parameters for Logistic Regression:',
      grid_search_logreg.best_params_)
print('Best Cross-Validation Accuracy:',
      grid_search_logreg.best_score_)

# Gunakan model terbaik
best_logreg = grid_search_logreg.best_estimator_

# Prediksi di test set
logreg_preds = best_logreg.predict(X_test)

# Evaluasi
print('Logistic Regression Performance on Test Set')
print('\n')
print(confusion_matrix(y_test, logreg_preds))
print('\n')
print(classification_report(y_test, logreg_preds))

```

gambar 4.2.3. implementasi logistic regression

- Multinomial logistic regression untuk klasifikasi multi-kelas
- Grid search untuk hyperparameter tuning:
 1. C: 0.01, 0.1, 1, 10, 100
 2. penalty: l1, l2

4. Naive Bayes

```

[ ] from sklearn.naive_bayes import GaussianNB
    from sklearn.metrics import confusion_matrix, classification_report

# Membuat model Gaussian Naive Bayes
nb = GaussianNB()

# Fit ke training data
nb.fit(X_train, y_train)

# Prediksi di test set
nb_preds = nb.predict(X_test)

# Evaluasi
print('Naive Bayes Performance on Test Set')
print('\n')
print(confusion_matrix(y_test, nb_preds))
print('\n')
print(classification_report(y_test, nb_preds))

```

gambar 4.2.4. implementasi naive bayes

Menggunakan Gaussian Naive Bayes

5. EVALUASI MODEL

a. Metrik Evaluasi

Untuk setiap model, digunakan confusion matrix dan classification report yang mencakup:

- Precision
- Recall
- F1-Score
- Support

b. Perbandingan Model

1. K-Nearest Neighbors

➡ K Nearest Neighbors K=16

```
[[62  6  2]
 [20  6  6]
 [ 9  3 22]]
```

	precision	recall	f1-score	support
0	0.68	0.89	0.77	70
1	0.40	0.19	0.26	32
2	0.73	0.65	0.69	34
accuracy			0.66	136
macro avg	0.60	0.57	0.57	136
weighted avg	0.63	0.66	0.63	136

gambar 5.2.1. Hasil evaluasi model KNN

- Akurasi : 66%
- Performance per kelas:
 - a. Kelas 0 (risiko rendah): Precision 0.68, Recall 0.89
 - b. Kelas 1 (risiko sedang): Precision 0.40, Recall 0.19
 - c. Kelas 2 (risiko tinggi): Precision 0.73, Recall 0.65

2. Decision Tree

```
Best Parameters: {'criterion': 'entropy', 'max_depth':  
20, 'min_samples_leaf': 4, 'min_samples_split': 2,  
'splitter': 'random'}  
Best Cross-Validation Accuracy: 0.709077380952381  
Decision Tree Performance on Test Set
```

```
[[65  3  2]  
 [19  8  5]  
 [ 5  8 21]]
```

	precision	recall	f1-score	support
0	0.73	0.93	0.82	70
1	0.42	0.25	0.31	32
2	0.75	0.62	0.68	34
accuracy			0.69	136
macro avg	0.63	0.60	0.60	136
weighted avg	0.66	0.69	0.66	136

gambar 5.2.2. Hasil implementasi decision tree

- Parameter terbaik: criterion=entropy, max_depth=20, min_samples_leaf=4, min_samples_split=2, splitter=random
- Akurasi validasi silang: 70.91%
- Akurasi test: 69%
- Performance per kelas:
 - a. Kelas 0 (risiko rendah): Precision 0.73, Recall 0.93
 - b. Kelas 1 (risiko sedang): Precision 0.42, Recall 0.25
 - c. Kelas 2 (risiko tinggi): Precision 0.75, Recall 0.62

3. Logistic Regression

➡ Best Parameters for Logistic Regression: {'C': 1, 'penalty': 'l1'}
Best Cross-Validation Accuracy: 0.6865079365079365
Logistic Regression Performance on Test Set

```
[[67  1  2]  
 [21  6  5]  
 [ 9  1 24]]
```

	precision	recall	f1-score	support
0	0.69	0.96	0.80	70
1	0.75	0.19	0.30	32
2	0.77	0.71	0.74	34
accuracy			0.71	136
macro avg	0.74	0.62	0.61	136
weighted avg	0.73	0.71	0.67	136

gambar 5.2.3. Hasil implementasi logistic regression

- Parameter terbaik: C=1, penalty=l1
- Akurasi validasi silang: 68.65%
- Akurasi test: 71%
- Performance per kelas:
 - a. Kelas 0 (risiko rendah): Precision 0.69, Recall 0.96
 - b. Kelas 1 (risiko sedang): Precision 0.75, Recall 0.19
 - c. Kelas 2 (risiko tinggi): Precision 0.77, Recall 0.71

4. Naive Bayes

➡ Naive Bayes Performance on Test Set

```
[[64  6  0]  
 [25  4  3]  
 [ 4  7 23]]
```

	precision	recall	f1-score	support
0	0.69	0.91	0.79	70
1	0.24	0.12	0.16	32
2	0.88	0.68	0.77	34
accuracy			0.67	136
macro avg	0.60	0.57	0.57	136
weighted avg	0.63	0.67	0.63	136

gambar 5.2.4. Hasil implementasi naive bayes

- Akurasi: 67%

- Performance per kelas:
 - a. Kelas 0 (risiko rendah): Precision 0.69, Recall 0.91
 - b. Kelas 1 (risiko sedang): Precision 0.24, Recall 0.12
 - c. Kelas 2 (risiko tinggi): Precision 0.88, Recall 0.68

6. ANALISIS HASIL

a. Perbandingan Performa Model

1. Akurasi Keseluruhan

Algoritma	Akurasi
Logistic Regression	71%
Decision Tree	69%
Naive Bayes	67%
KNN	66%

2. Performa Kelas “Low Risk” (0):

- Semua model memiliki recall tinggi untuk kelas ini
- Decision Tree memiliki precision tertinggi (0.73)
- Logistic Regression memiliki recall tertinggi (0.96)

3. Performa Kelas “Mid Risk” (1):

- Semua model kesulitan mengklasifikasikan kelas ini dengan baik
- Logistic Regression memiliki precision tertinggi (0.75)
- Decision Tree memiliki recall tertinggi (0.25)

4. Performa Kelas “High Risk” (2):

- Naive Bayes memiliki precision tertinggi (0.88)
- Logistic Regression memiliki recall tertinggi (0.71)

b. Analisis Hasil Tiap Algoritma

1. K-Nearest Neighbors

- Kelebihan: Performa baik untuk kelas risiko rendah
- Kekurangan: Buruk dalam mengklasifikasikan risiko sedang
- Insight: Model KNN dengan $K=16$ lebih sesuai untuk mengidentifikasi kasus risiko rendah daripada kasus risiko sedang atau tinggi

2. Decision Tree

- Kelebihan: Performa baik dan seimbang untuk kelas risiko rendah dan tinggi
- Kekurangan: Precision masih rendah untuk kelas risiko sedang
- Insight: Decision Tree dengan parameter optimal (entropy, max_depth=20) efektif mengidentifikasi kasus risiko rendah

3. Logistic Regression

- Kelebihan: Akurasi tertinggi dengan performa seimbang di semua kelas
- Kekurangan: Recall rendah untuk kelas risiko sedang
- Insight: Model ini memberikan keseimbangan terbaik secara keseluruhan antara semua kelas

4. Naive Bayes

- Kelebihan: Precision sangat tinggi untuk kelas risiko tinggi (0.88)
- Kekurangan: Sangat buruk untuk mengklasifikasikan risiko sedang
- Insight: Model ini sangat baik untuk mengidentifikasi kasus risiko tinggi dengan keyakinan tinggi

7. KESIMPULAN DAN REKOMENDASI

a. Kesimpulan

1. Logistic Regression memberikan kinerja terbaik secara keseluruhan dengan akurasi 71% dan keseimbangan yang baik antara precision dan recall di semua kelas.
2. Semua model mengalami kesulitan dalam mengklasifikasikan risiko sedang (kelas 1), yang menunjukkan bahwa fitur-fitur yang ada mungkin tidak sepenuhnya dapat membedakan kelas ini dari kelas lainnya.
3. Untuk penggunaan klinis:
 - Jika tujuannya adalah mengidentifikasi kasus risiko tinggi dengan tingkat kepercayaan yang tinggi, Naive Bayes adalah model terbaik.
 - Jika membutuhkan keseimbangan antara semua kelas risiko, Logistic Regression adalah pilihan terbaik.

b. Rekomendasi

1. Pengumpulan Data Tambahan: Menambah data untuk kelas risiko sedang untuk meningkatkan kemampuan model dalam mengklasifikasikan kelas ini.
2. Feature Engineering: Menambahkan atau mengeksplorasi fitur-fitur baru yang dapat membantu membedakan kelas risiko sedang dengan lebih baik.
3. Pendekatan Ensemble: Menggunakan pendekatan ensemble seperti Random Forest atau voting classifier untuk menggabungkan kekuatan dari beberapa model.
4. Implementasi Praktis: Untuk penggunaan klinis, disarankan menggunakan:
 - Naive Bayes jika prioritas adalah identifikasi risiko tinggi dengan akurasi maksimal
 - Logistic Regression jika dibutuhkan keseimbangan di semua tingkat risiko
5. Optimasi Lebih Lanjut: Melakukan hyperparameter tuning lebih lanjut dan teknik-teknik seperti class balancing untuk meningkatkan performa model, terutama untuk kelas risiko sedang.

Dengan model prediktif yang tepat, deteksi dini risiko maternal dapat ditingkatkan, sehingga memungkinkan intervensi medis yang tepat waktu dan berpotensi menyelamatkan nyawa ibu hamil dan bayi mereka